

Full Stack Developer Assignment: Task Management App

Objective: Develop a full-stack task management application using the MERN (MongoDB, Express.js, React.js/Next.js, Node.js) stack. The application should allow users to create, update, delete, and organize tasks effectively. The focus is on demonstrating your skills with MongoDB, Express.js, React / NEXTJS, Node.js, and overall full-stack development best practices.

Requirements:

1. Basic Features:

- **Task Creation:** Users should be able to create a new task with title, description, due date, and priority level (e.g., High, Medium, Low). The task should be stored in a MongoDB database.
- **Task List:** Display a list of tasks with sorting options based on due date and priority. The list should be fetched from the backend API.
- **Task Details:** Users should be able to view details of each task in a separate view or modal. Data should be retrieved from the backend.
- **Task Editing:** Users should be able to edit task details. The edited task should be updated in the MongoDB database.
- **Task Deletion:** Users should be able to delete tasks, and this action should remove the task from the MongoDB database.

2. Analytical Dashboard:

Overview: Create an analytical dashboard that provides insights into the tasks, use **MongoDB Aggregation** for real-time analytics.

Visualizations:

- **Task Distribution:** A chart (e.g., pie chart or bar graph) showing the distribution of tasks based on priority levels.
- **Completion Rate:** A line graph or progress bar showing the percentage of tasks completed over time.
- **Upcoming Deadlines:** A list or calendar view highlighting tasks with upcoming deadlines.

3. Backend Development:

- **RESTful API:** Develop a RESTful API using Node.js and Express.js to handle CRUD operations (Create, Read, Update, Delete) for tasks.
- **MongoDB Integration:** Use MongoDB as a database to store and manage tasks. Implement Mongoose for schema management.
 - Implement **MongoDB Aggregation** for optimized filtering and dashboard analytics.
 - Use **Mongoose** to define a Task schema with appropriate validations.
- **API Endpoints:**
 - POST /tasks: Create a new task.
 - GET /tasks: Retrieve a list of tasks with options for filtering and sorting.
 - GET /tasks/:id: Retrieve the details of a specific task.
 - PUT /tasks/:id: Update a specific task.
 - DELETE /tasks/:id: Delete a specific task.

4. Advanced Features:

- **Task Completion:** Allow users to mark tasks as completed. Update the task's status in the database.
- **User Authentication (JWT): -**
 - Implement secure login/signup using JWT authentication.
 - Password hashing with bcrypt for security.
 - Store tokens securely (e.g., httpOnly cookies).
- **Task Filtering:** Implement filtering options on both the frontend and backend to show only completed or pending tasks.
- **Search Functionality:** Include a search bar to filter tasks by title or description.
- **Authentication (Optional):** Implement user authentication (e.g., JWT) to secure the task management features.
- **Responsive Design:** Ensure the app is responsive and works well on various devices.

5. UI/UX:

- **Design:** Implement a clean and user-friendly interface using a UI framework like Material-UI, Bootstrap, or custom styling.
- **Validation:** Implement form validation with error messages to ensure all required fields are filled out when creating or editing a task.

6. Code Quality:

- **Component Structure:** Organize frontend components in a logical and maintainable manner, Use **reusable components** for forms, lists, and modals.
- **API Structure:** Structure your backend code with a clear separation of concerns (e.g., controllers, services, models).
- **Backend Structure (MVC Architecture):**

/backend

- |— **controllers/ (Handles business logic)**
- |— **models/ (Defines MongoDB schemas)**
- |— **routes/ (Defines API endpoints)**
- |— **middleware/ (Authentication & security)**
- |— **config/ (Database connection, environment variables)**

- **Documentation:** Write clear comments and documentation where necessary, including a README.md file in the GitHub repository.
- **Error handling & Validation:**
 - Use **express-validator** for validating API inputs.
 - Implement error-handling and authorization middleware's to return user-friendly errors and maintain security.
- **Best Practices:** Follow best practices for the MERN stack, **Helmet.js** for securing HTTP headers. including state management with React (or Redux), secure API development, and efficient database interaction.

Deliverables:

- A GitHub repository containing the source code.
- **Frontend (React/Next.js)** and **Backend (Node.js/Express).**
- Setup & installation instructions

- API documentation
- A README.md file with setup instructions, explaining how to run both the frontend and backend.

Submission Deadline:

- Please complete and submit the assignment within 3 days.

Evaluation Criteria:

- **Functionality:** How well does the app meet the requirements?
- **Code Quality:** Are the code clean, well-organized, and following best practices?
- **UI/UX:** Is the user interface intuitive and visually appealing?
- **Backend Implementation:** Are **MongoDB, Express.js, and API security** implemented efficiently?
- **Analytical Dashboard:** How well does the dashboard visualize and present data?
- **Creativity:** Any additional features or innovations?